

Lectura 8.1: Herramientas avanzadas y misceláneos

Introducción a las redes neuronales informadas por física
Especialización en Inteligencia Artificial - B32025

Outline

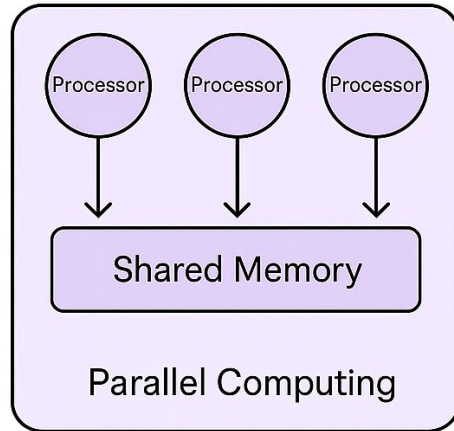
- cPINN y XPINN
- PINN con arquitecturas de CNN, RNN, LSTM, etc
- Frameworks: DeepXDE, NVIDIA Modulus
- Comentarios finales

- Costo computacional del entrenamiento de vanilla PINN (y sus variantes vistas hasta ahora) es mayor a medida que aumenta la complejidad del problema
- Posible solución? realizar cómputo paralelo/distribuido
- Por ejemplo, utilizando backend de GPU en Colab ya estamos haciendo cómputo paralelo... Es suficiente?

cPINN y XPINN - Paradigmas de cómputo

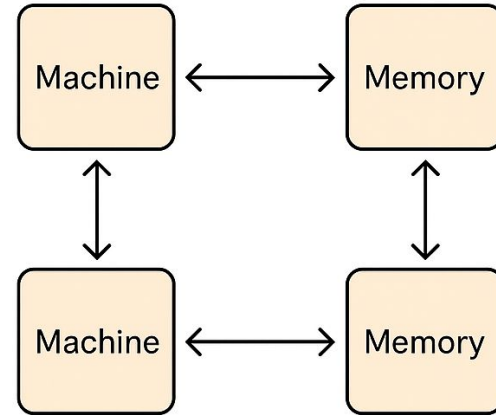
Parallel Computing

- Multiple processors
- Single machine
- Shared memory



Distributed Computing

- Multiple computers
- Network
- Distributed memory



MPI (Message Passing Interface)

cPINN y XPINN - Descomposición de dominio

- Técnica de “Divide and Conquer” (y “Recombine”) aplicada a un problema definido sobre un dominio, que consiste en la solución de subproblemas relacionados que se resuelven sobre subdominios no superpuestos
- Suposiciones:
 - La solución de los subproblemas es cuantitativa o cualitativamente “más fácil” que el problema original
 - El problema original no se ajusta en el espacio computacional disponible
 - Los subproblemas pueden ser resueltos con algún grado de concurrencia

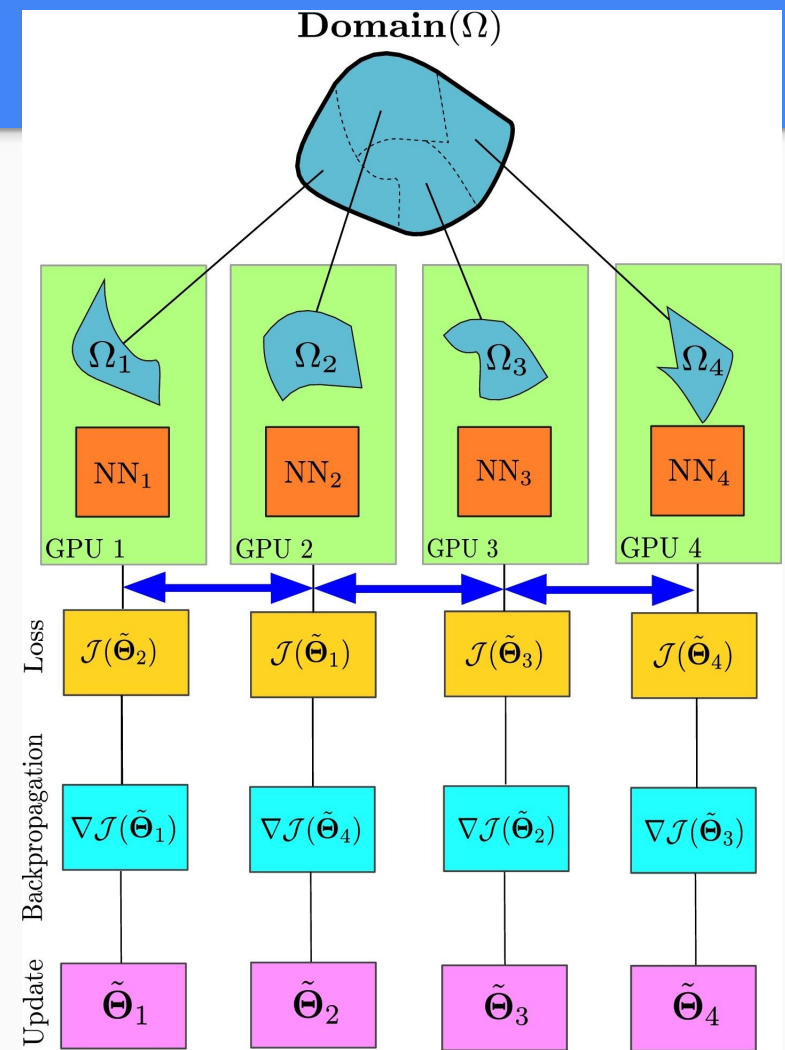
cPINN y XPINN

- cPINN: aplicado a ecuaciones de conservación. Permite asegurar la continuidad del flujo entre soluciones locales en las interfaces.
- XPINN: permite encodear otras ecuaciones, no necesariamente de conservación
- En ambos, la obtención de la solución se realiza en subdominios, y luego se comunican las soluciones entre vecinos
- Usos: problemas multiescala; problemas con características no suaves (ondas de choque, fisuras, etc); problemas multifísica, etc.

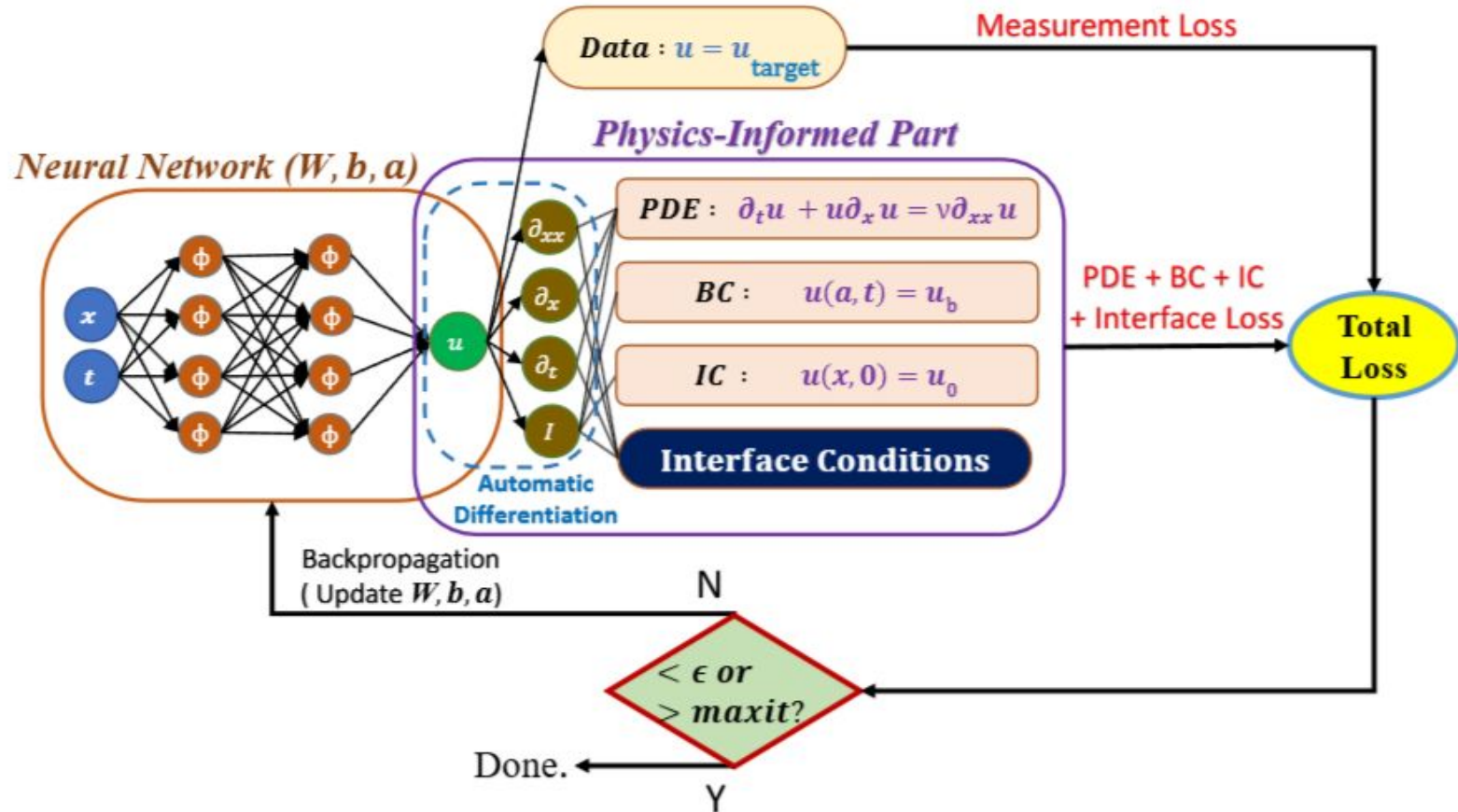
cPINN y XPINN

- cPINN: descomposición de dominio en dominio espacial
- XPINN: descomposición de dominio en dominio espacial-temporal
- Técnicas similares al paradigma “data-parallel”, pero con diferentes NN
- Una función de pérdida por cada subdominio
- Condiciones adicionales en las interfaces entre dominios adyacentes

Shukla, Jagtap & Karnadiakis. **Parallel physics-informed neural networks via domain decomposition**. *Journal of Computational Physics* (2021).

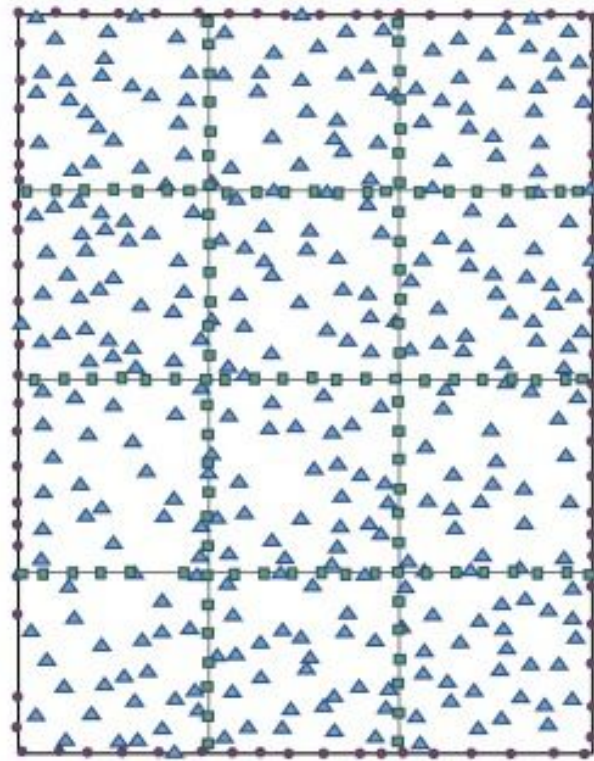


Arquitectura XPINN

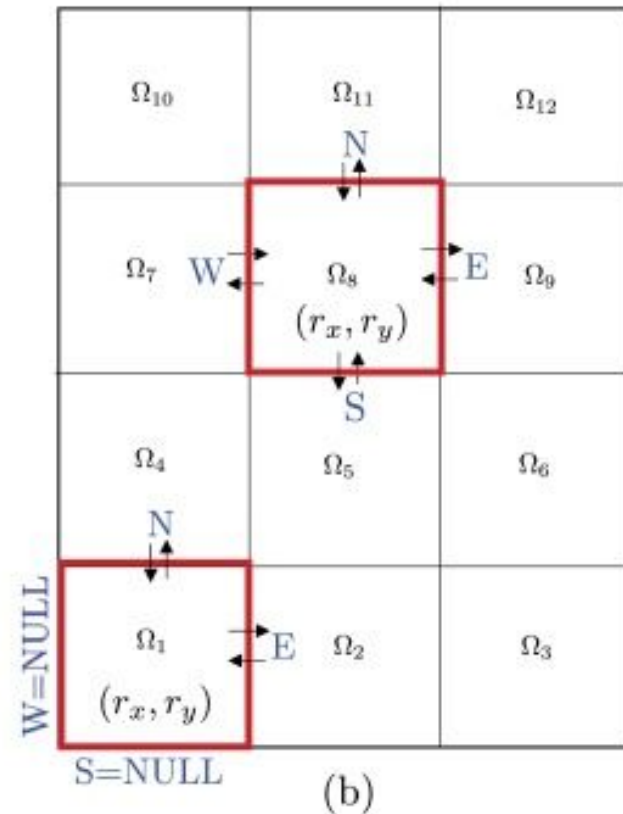


Puntos de colocación

▲ Residual Points • Data Points ■ Interface Points



(a)



(b)

XPINN - Ejemplo: ecuación de Burgers viscosa

Subdomain number	1	2
# Layers	6	7
# Neurons	20	25
# Residual points	7000	3000
Adaptive Activation function	tanh	sin

$$\mathcal{J}(\tilde{\Theta}_1) = W_{u_1} \text{MSE}_{u_1} + W_{\mathcal{F}_1} \text{MSE}_{\mathcal{F}_1}, (\text{Subdomain 1}),$$

$$\mathcal{J}(\tilde{\Theta}_2) = W_{\mathcal{F}_2} \text{MSE}_{\mathcal{F}_2}, (\text{Subdomain 2}),$$

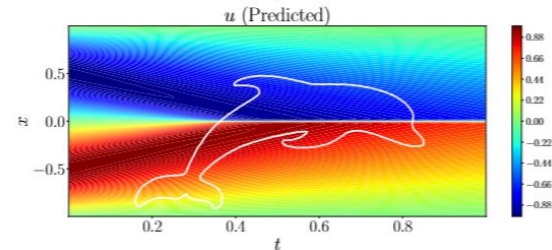
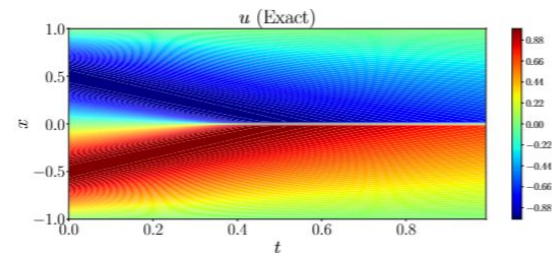
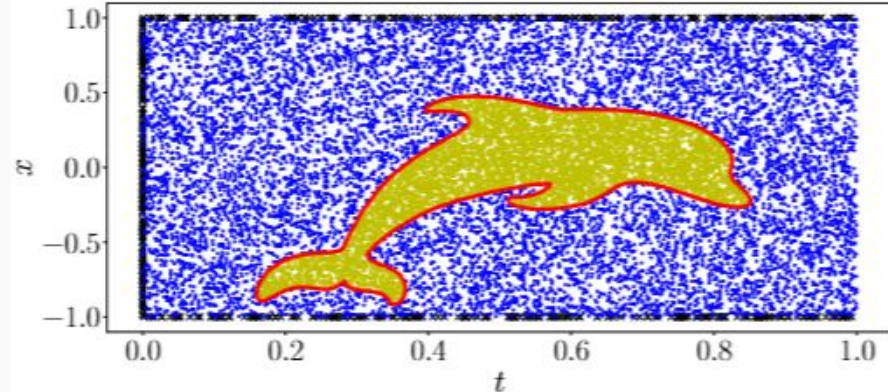
$$\mathcal{J}(\tilde{\Theta}_{\text{Interface}}) = W_{I_{\mathcal{F}_1}} \text{MSE}_{\mathcal{R}} + W_{I_1} \text{MSE}_{u_{avg}}, (\text{Interface}).$$

$$\text{MSE}_{u_{avg}}(\tilde{\Theta}_q; \{\mathbf{x}_{I_q}^{(i)}\}_{i=1}^{N_{I_q}}) =$$

$$\sum_{\forall q^+} \left(\frac{1}{N_{I_q}} \sum_{i=1}^{N_{I_q}} \left| u_{\tilde{\Theta}_q}(\mathbf{x}_{I_q}^{(i)}) - \left\{ \left\{ u_{\tilde{\Theta}_q}(\mathbf{x}_{I_q}^{(i)}) \right\} \right\} \right|^2 \right),$$

$$\text{MSE}_{\mathcal{R}}(\tilde{\Theta}_q; \{\mathbf{x}_{I_q}^{(i)}\}_{i=1}^{N_{I_q}}) =$$

$$\sum_{\forall q^+} \left(\frac{1}{N_{I_q}} \sum_{i=1}^{N_{I_q}} \left| \mathcal{F}_{\tilde{\Theta}_q}(\mathbf{x}_{I_q}^{(i)}) - \mathcal{F}_{\tilde{\Theta}_{q^+}}(\mathbf{x}_{I_q}^{(i)}) \right|^2 \right),$$



Modelos fundacionales para ciencia e ingeniería

- **Youngsoo Choi et al. (2025, pre-print): Defining Foundation Models for Computational Science: A Call for Clarity and Rigor**
- Inspirados en el éxito de los foundation models en NLP y visión por computadora.
- Uso creciente del término en Ciencia Computacional, **muchas veces sin una definición clara.**
- Objetivo: proponer una definición formal y criterios de evaluación.
- Ejemplo propuesto: Data-Driven Finite Element Method (DD-FEM).

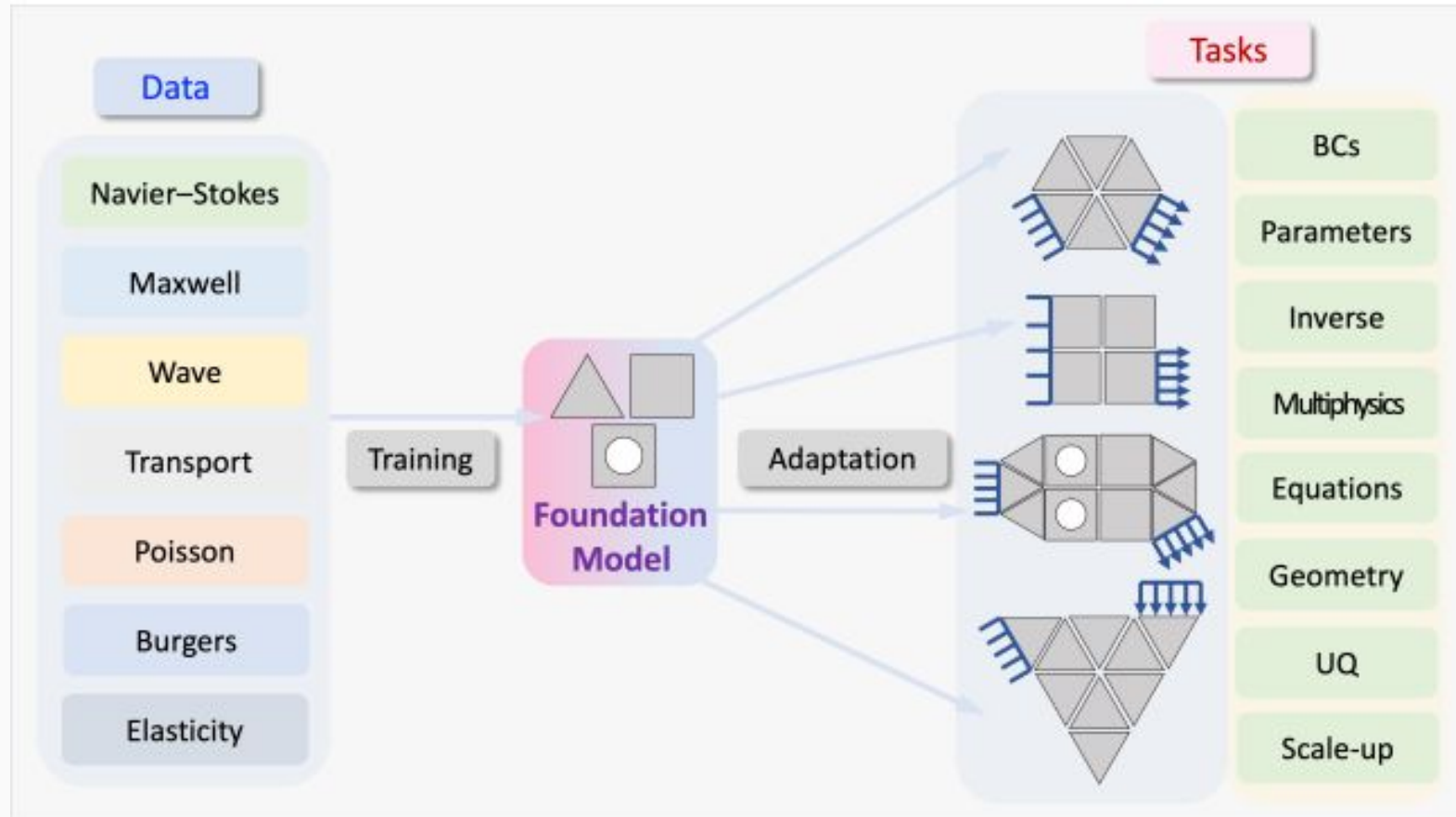
Definición propuesta:

Un modelo fundacional en ciencia computacional es un modelo basado en datos, entrenado sobre una amplia distribución de tipos de aplicaciones científicas o sistemas físicos, que presenta una amplia capacidad de generalización a través de problemas científicos, dominios computacionales, tareas y condiciones físicas, sin requerir reentrenamiento desde cero ni modificaciones estructurales, y que sirve como una base reutilizable.

Modelos fundacionales para ciencia e ingeniería

- Capacidad para aceptar y procesar datos científicos multimodales (mediciones de sensores, salidas de simulaciones de diferentes EDP o dominios físicos, descripciones de materiales)
- Simulaciones monofísicas
- Simulaciones multifísicas: Modelado de sistemas acoplados como interacción fluido–estructura, etc, integrando entradas y salidas multimodales en distintos dominios físicos.
- Problemas inversos y estimación de parámetros: Inferencia de parámetros desconocidos, usando datos heterogéneos (sensores, campo, simulaciones de distintas EDP).
- Cuantificación de incertidumbre: Estimación de variabilidad o intervalos de confianza en predicciones
- Predicción y control en tiempo real: Uso en entornos críticos como lazos de retroalimentación con sensores o sistemas de control en línea.
- Optimización de diseño: Sustituto rápido y preciso de simulaciones de alta fidelidad en flujos iterativos de optimización.

Modelos fundacionales para ciencia e ingeniería



Modelos fundacionales para ciencia e ingeniería

Características deseables:

- Consistencia científica: respetar leyes físicas y simetrías.
- Extrapolación robusta en espacio y tiempo.
- Escalabilidad con datos: mejora con datasets más diversos.
- Rigor matemático: análisis de estabilidad, convergencia y error.

Modelos fundacionales para ciencia e ingeniería

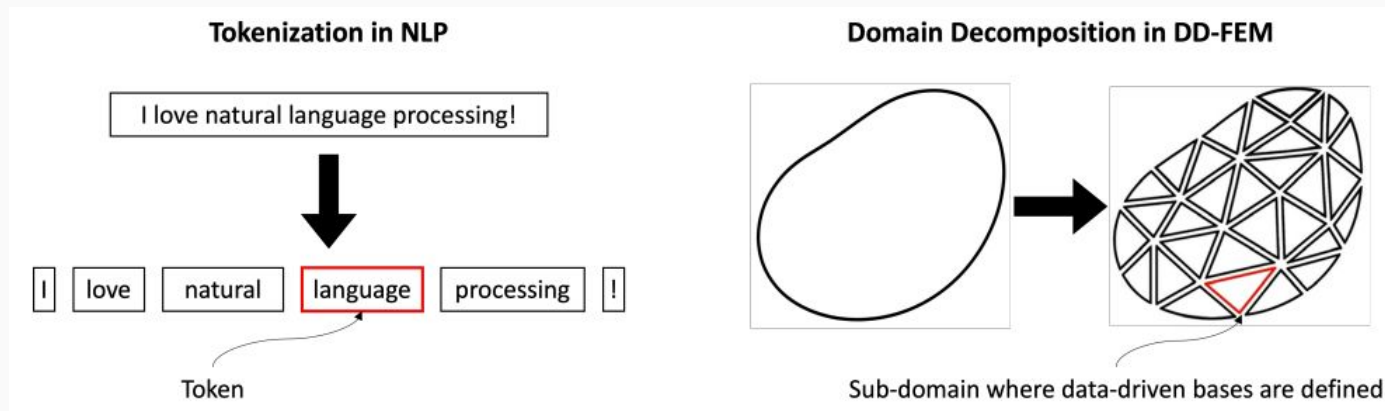
Desafíos para lograr estos modelos:

- Tamaño masivo de datos por simulación.
- Costos altos de fine-tuning en nuevos escenarios.
- Heterogeneidad y falta de datasets estandarizados.
- Dependencia de mallas y geometrías específicas.
- Integrar restricciones físicas sin perder generalidad
- Multi-escalabilidad y comportamiento a tiempos largos

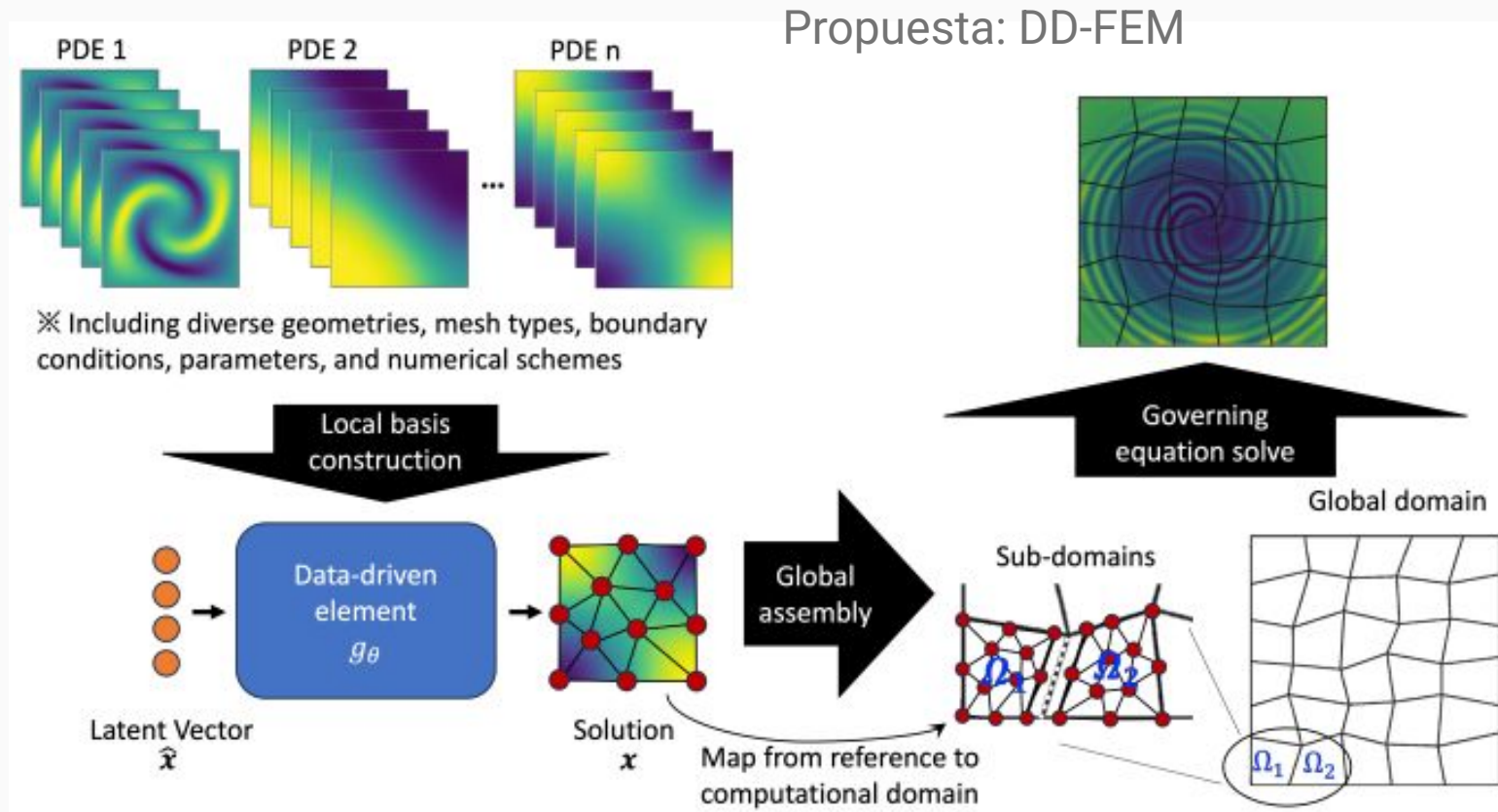
Modelos fundacionales para ciencia e ingeniería

Propuesta: DD-FEM

- Estructura inspirada en FEM clásico.
- Reemplaza bases polinomiales por bases aprendidas localmente.
- Ensamblaje global que respeta leyes físicas.
- Permite reducir datos, reutilizar componentes y generalizar a dominios grandes.



Modelos fundacionales para ciencia e ingeniería



PINN con arquitecturas diferentes. Ejemplo: PhyCRNet

- Ren, P., Rao, C., Liu, Y., Wang, J. X., & Sun, H. (2022). PhyCRNet: Physics-informed convolutional-recurrent network for solving spatiotemporal PDEs. *Computer Methods in Applied Mechanics and Engineering*, 389, 114399.
- Combina arquitecturas convoluciones y recurrentes para resolver PDEs sin datos rotulados
- Utiliza red neuronal convolucional de encoder-decoder con memoria de corto-largo plazo (ConvLSTM)
- Permite naturalmente la imposición “hard” de CB tanto Dirichlet como Neumann
- Evolución temporal simulada explícitamente mediante conexiones residuales y autoregresivas

PhyCRNet

PDE Residual $\mathcal{R} = \mathbf{u}_t + \mathcal{F}(\mathbf{u}, \nabla_{\mathbf{x}} \mathbf{u}, \nabla_{\mathbf{x}}^2 \mathbf{u}, \dots) \rightarrow 0$

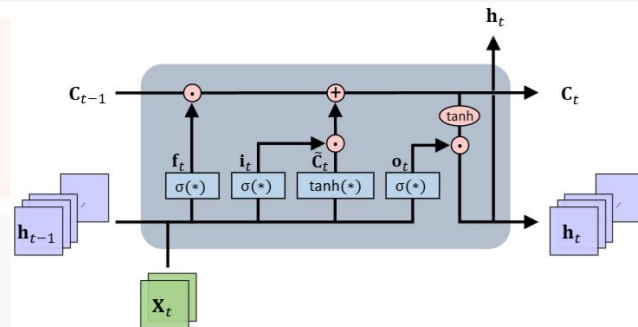
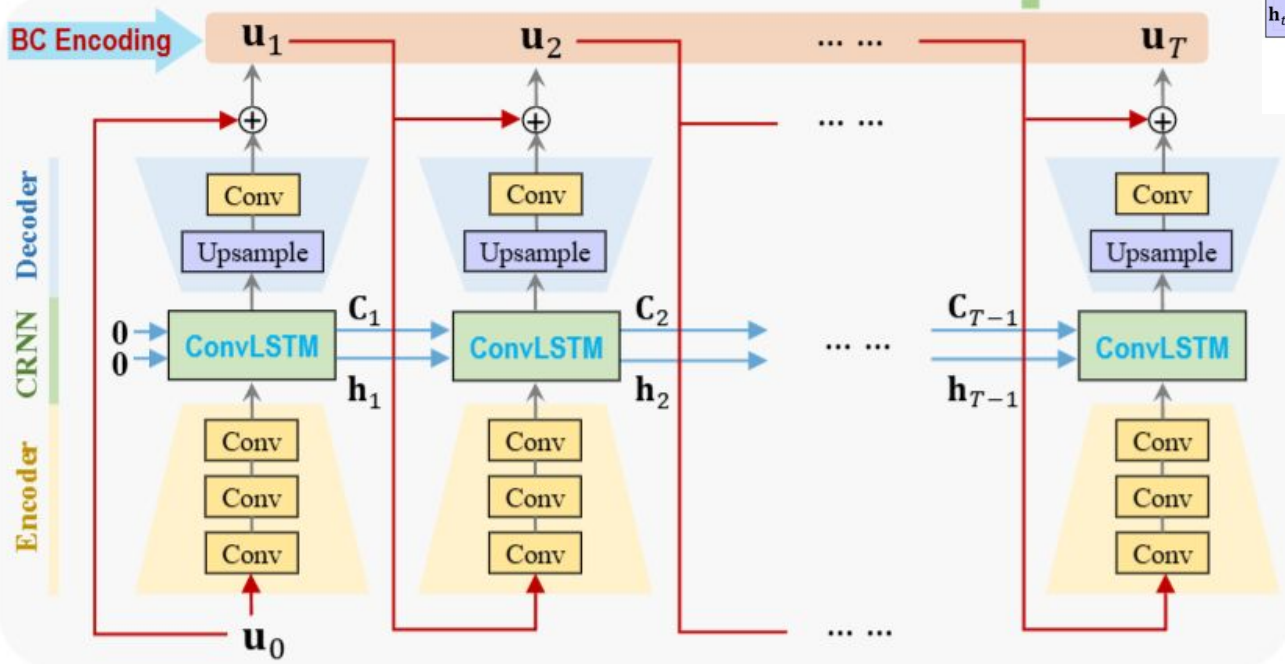
Loss Function: $\mathcal{L}_p(\boldsymbol{\theta}) = \sum \|\mathcal{R}\|_2^2$

Solution: $\boldsymbol{\theta}^* = \arg \min_{\boldsymbol{\theta}} \mathcal{L}_p(\boldsymbol{\theta})$

$\mathbf{u}, \mathbf{u}_t, \nabla_{\mathbf{x}} \mathbf{u}, \nabla_{\mathbf{x}}^2 \mathbf{u} \dots$

Differentiation

Finite Difference-based Filters



Arg. ConvLSTM:
 $\mathbf{h}$: "estado oculto";
 $\mathbf{C}$: "estado de celda"

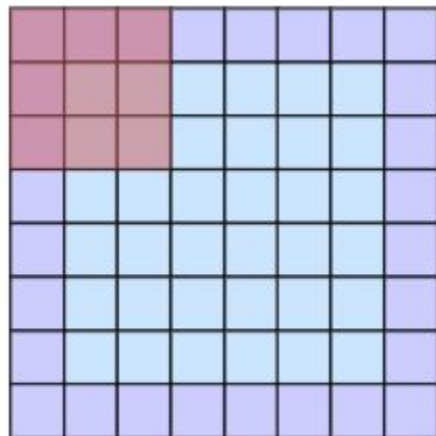
- Encoder: incluye 3 capas convolucionales (con funciones de activación ReLU) que permiten el aprendizaje de características latentes de baja dimensionalidad del estado input “u”
- ConvLSTM: actúa como un propagador temporal de dichas características de baja dimensionalidad
- Decoder: reconstrucción del espacio latente de baja resolución a uno de alta resolución mediante operación de upsampling.
- Conexión residual inspirada por el método de Euler: $\mathbf{u}_{i+1} = \mathbf{u}_i + \delta t \cdot \mathcal{NN}[\mathbf{u}_i; \theta]$
- La arquitectura comunica no sólo el espacio latente, sino también las entradas y salidas “u_i”, lo cual permitiría tomar pasos de tiempo más largos

PhyCRNet - Diferenciación temporal/espacial

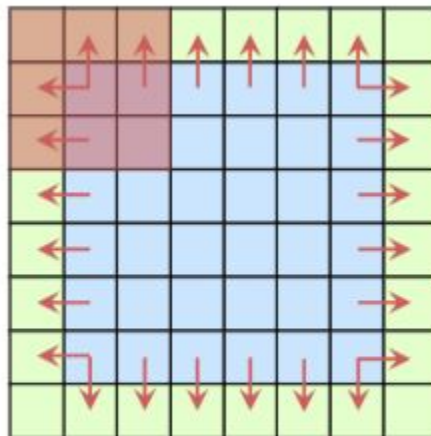
- Dataset de entrenamiento: puntos de colocación ubicados en el espacio (recordar “estrategias de sampleo”)
- Espacio solución: se construye sobre una discretización regular tipo grilla

Filtro de FD de 2o orden: $K_t = [-1, 0, 1] \times \frac{1}{2\delta t},$

Filtro de FD de 4o orden: $K_s = \begin{bmatrix} 0 & 0 & -1 & 0 & 0 \\ 0 & 0 & 16 & 0 & 0 \\ -1 & 16 & -60 & 16 & -1 \\ 0 & 0 & 16 & 0 & 0 \\ 0 & 0 & -1 & 0 & 0 \end{bmatrix} \times \frac{1}{12(\delta x)^2}$



Dirichlet BCs

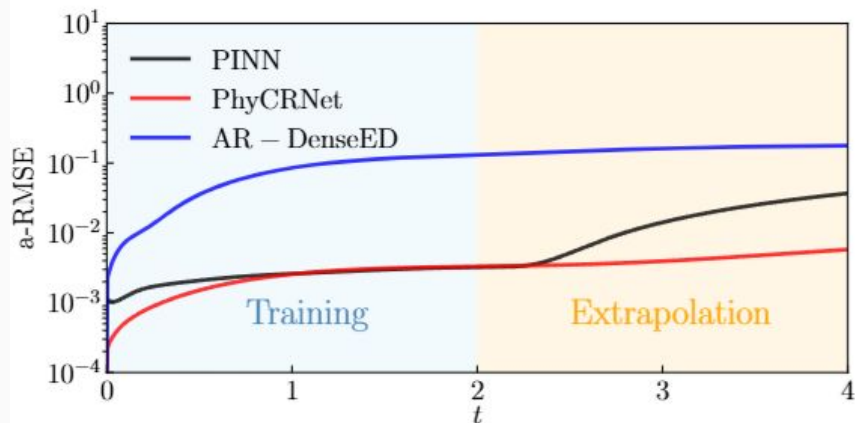


Neumann BCs

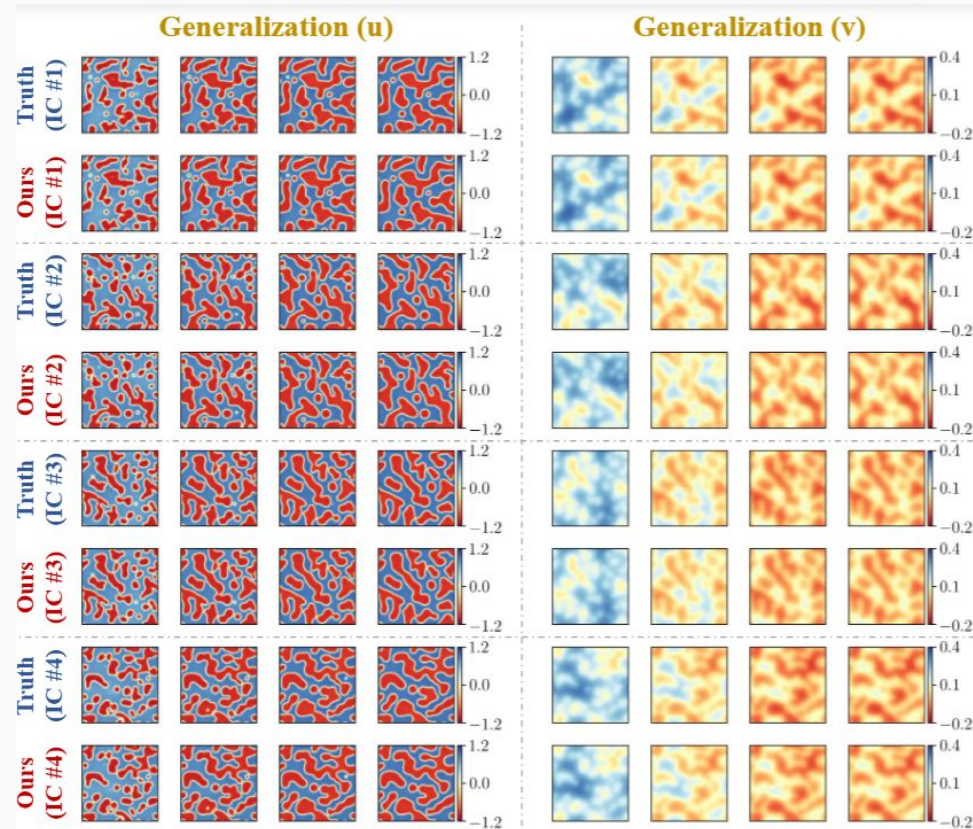
-  Internal nodes
-  Constant nodes
-  Ghost nodes
-  Conv filter

PhyCRNet - Algunos ejemplos

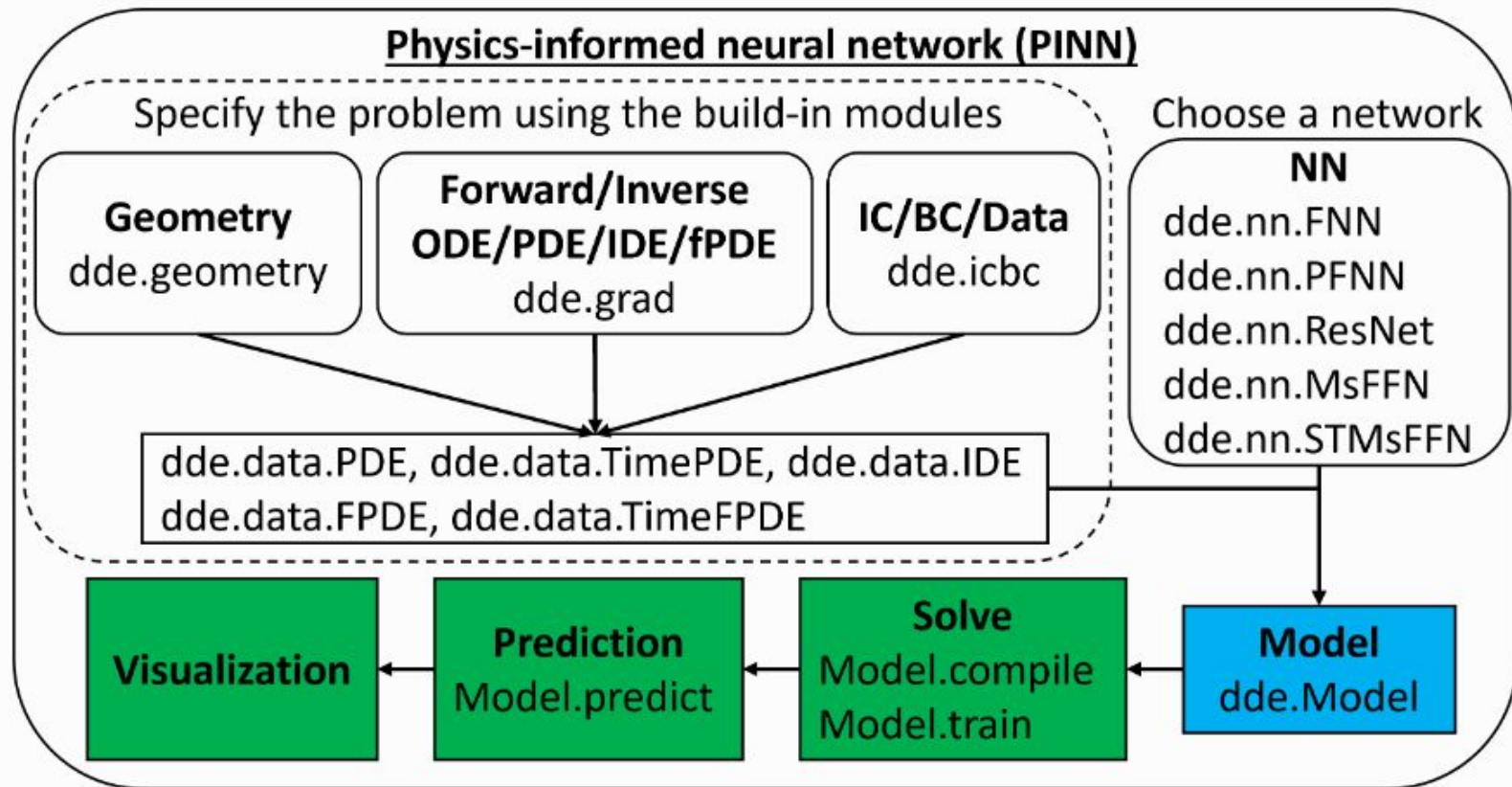
Extrapolación - Ec. de Burgers viscosa

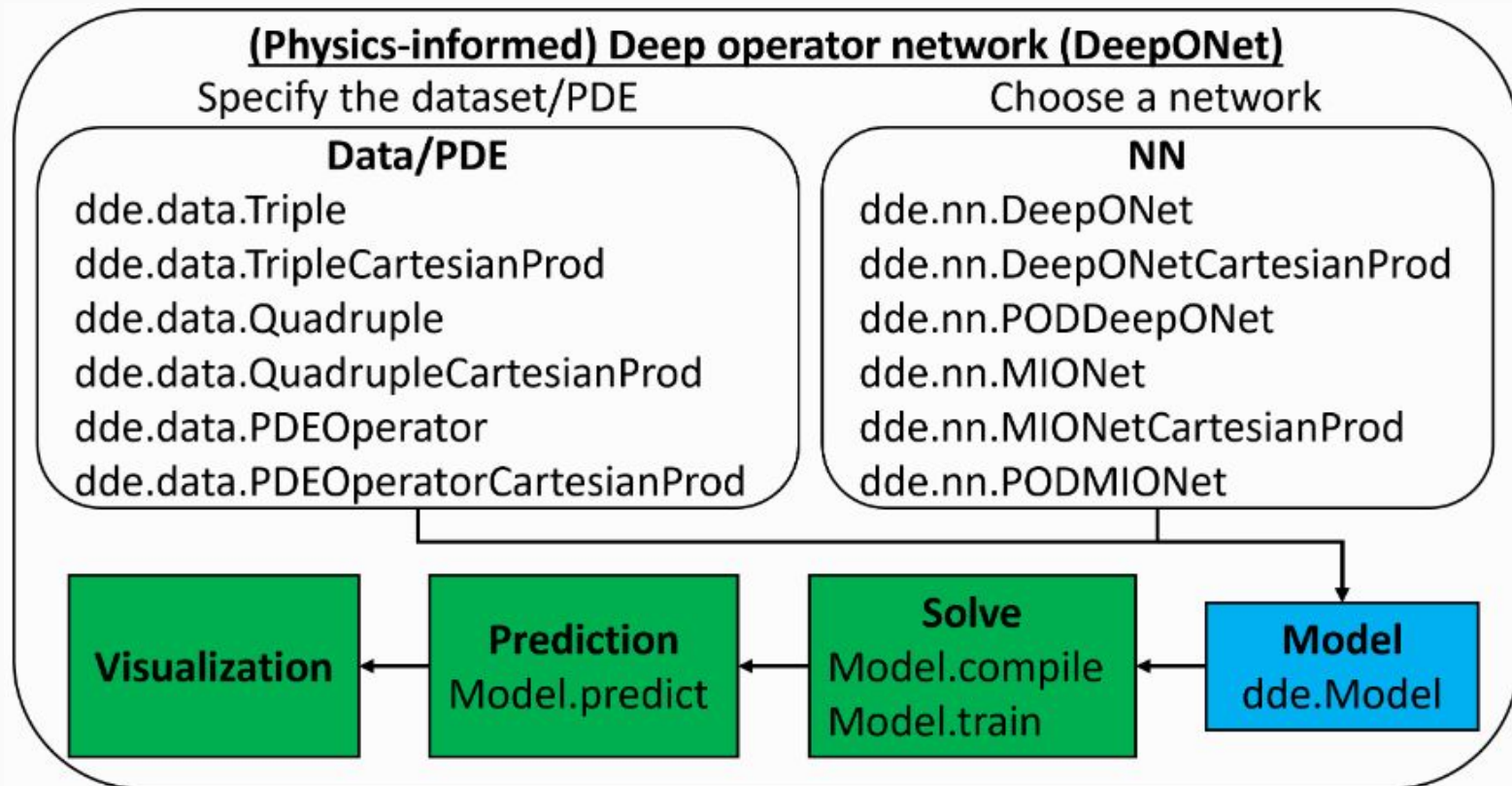


Diferentes IC - Ec. de FN RD



- Librería de Scientific Machine Learning, puntualmente para resolver PDEs
- <https://deepxde.readthedocs.io/en/latest/>
- Interfaz mediante Python
- Incluye los siguientes algoritmos/métodos:
 - PINN (problemas directos, inversos, fraccionales, etc)
 - DeepONets y PI-DeepONets
 - Redes neuronales multi-fidelidad
- Backends que soports: TF 1.x, TF 2.x, PyTorch, JAX





DeepXDE - Características

- La implementación trata de imitar la formulación matemática
- Permite la creación de dominios complejos a través de geometrías primitivas y operaciones booleanas
- Implementa 5 tipos de BC: Dirichlet, Neumann, Robin, periódica, y general
- Varios métodos de sampleo fijos (LHS, Sobol, Hammersley, etc) y de métodos adaptativos
- Posibilidad de hacer cómputo distribuido en múltiples GPUs
- Varios callbacks para monitorear el entrenamiento
- Varios tipos de pérdidas, learning rate schedules, métricas, etc

NVIDIA PhysicsNeMo (ex Modulus)

- Framework open-source para construir, entrenar y hacer fine-tuning the modelos Physics-ML con interfaz de Python
- <https://developer.nvidia.com/modulus>
- <https://github.com/NVIDIA/modulus/tree/main>
- Cuenta con una comunidad activa que permite contar con modelos y métodos SOTA
- Integración directa y sencilla con PyTorch, lo que permite a los usuarios de esta librería contar con un entorno amigable y conocido
- Integración con NVIDIA Omniverse: plataforma para conectar y construir pipelines 3D para visualizar los resultados obtenidos con Modulus
- Permite obtener resultados en tiempo cuasi-real → Gemelos digitales

NVIDIA PhysicsNeMo (ex Modulus)



AI Toolkit for Physics

Quickly configure, build, and train AI models for physical systems in any domain, from engineering simulations to life sciences, with simple Python APIs.



Customize Models

Download, build on, and customize state-of-the-art pretrained models from the [NVIDIA NGC™ catalog](#).



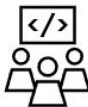
Near-Real-Time Inference

Deploy AI surrogate models as digital twins of your physical systems to simulate in near real time.



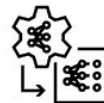
Scale With NVIDIA AI

Leverage NVIDIA AI to scale training performance from a single GPU to multi-node implementations.



Open-Source Design

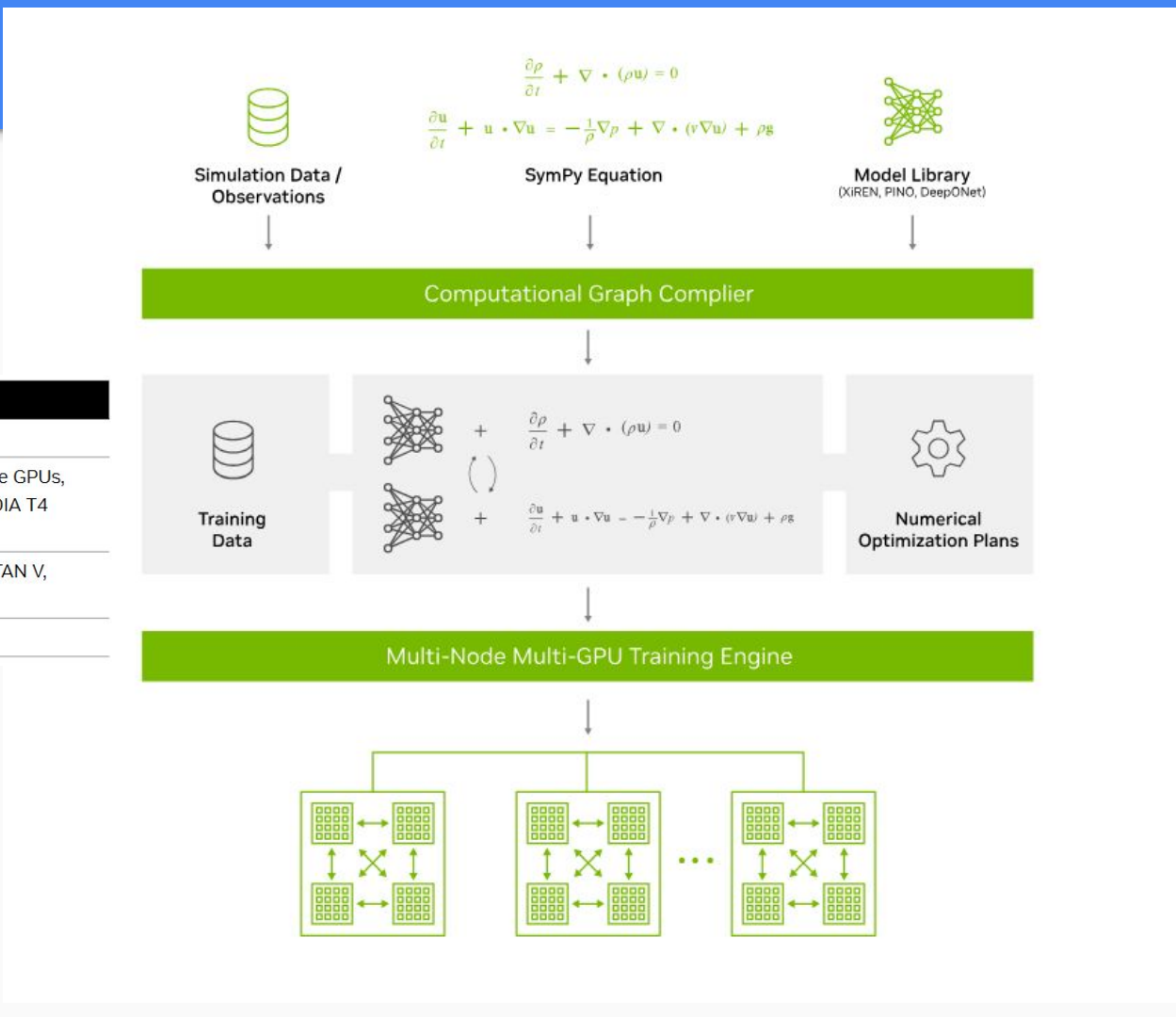
Experience the benefits of open source. PhysicsNeMo is built on top of PyTorch and is released under the Apache 2.0 license.



Standardized

Work with the best practices of AI development for physics-ML models, with an immediate focus on engineering applications.

NVIDIA Modulus



Hardware Requirements	
CPU	64-bit x86
GPU	NVIDIA A100, A30, A4000 Tensor Core GPUs, NVIDIA V100 Tensor Core GPUs, NVIDIA T4 Tensor Core GPUs
Other supported GPUs	NVIDIA RTX™ 30x GPUs, NVIDIA TITAN V, Quadro® GV100
System memory	Minimum 16GB per GPU

CPU	64-bit x86
GPU	NVIDIA A100, A30, A4000 Tensor Core GPUs, NVIDIA V100 Tensor Core GPUs, NVIDIA T4 Tensor Core GPUs
Other supported GPUs	NVIDIA RTX™ 30x GPUs, NVIDIA TITAN V, Quadro® GV100
System memory	Minimum 16GB per GPU

NVIDIA Modulus

Modulus Core

Distributed Utilites

Efficient data parallel training
across multi-GPU and multi-node

Data Pipelines

GPU-accelerated data loading and
pre-processing

Models

Collection of state-of-the-art deep
learning models for Phys-ML

Performance Utilites

Amp, JIT, Cuda Graphs

Modulus Launch

Model Checkpointing

Seamless saving, loading, enabling
custom model registration and
direct model instantiation

Logging Monitoring

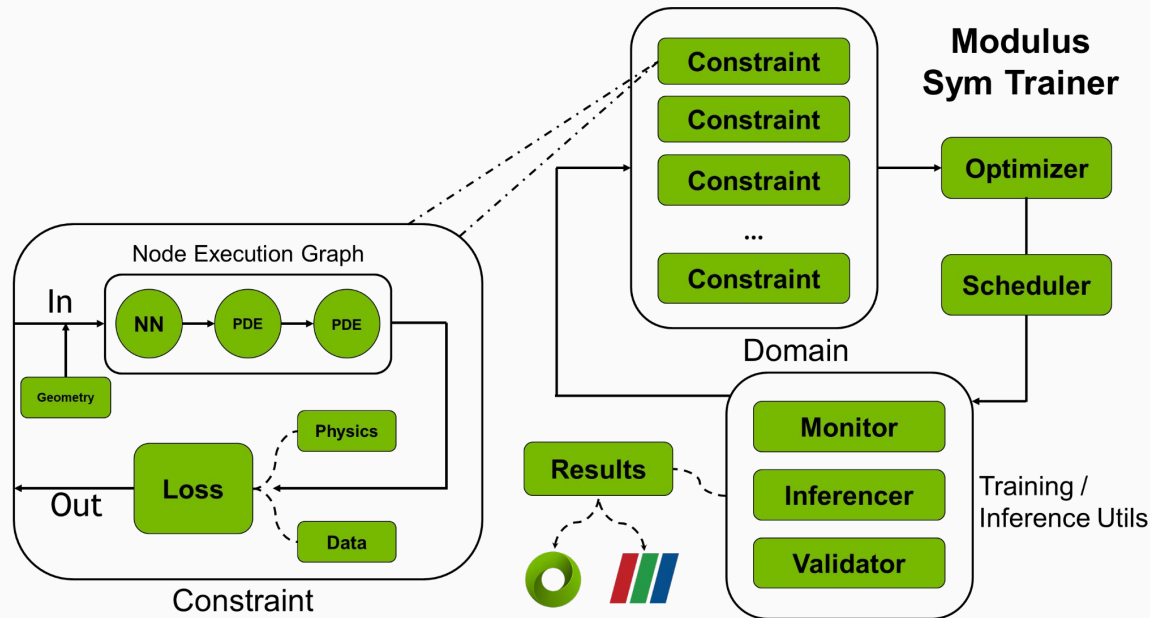
Tensorboard, W&B, MLFlow

Examples

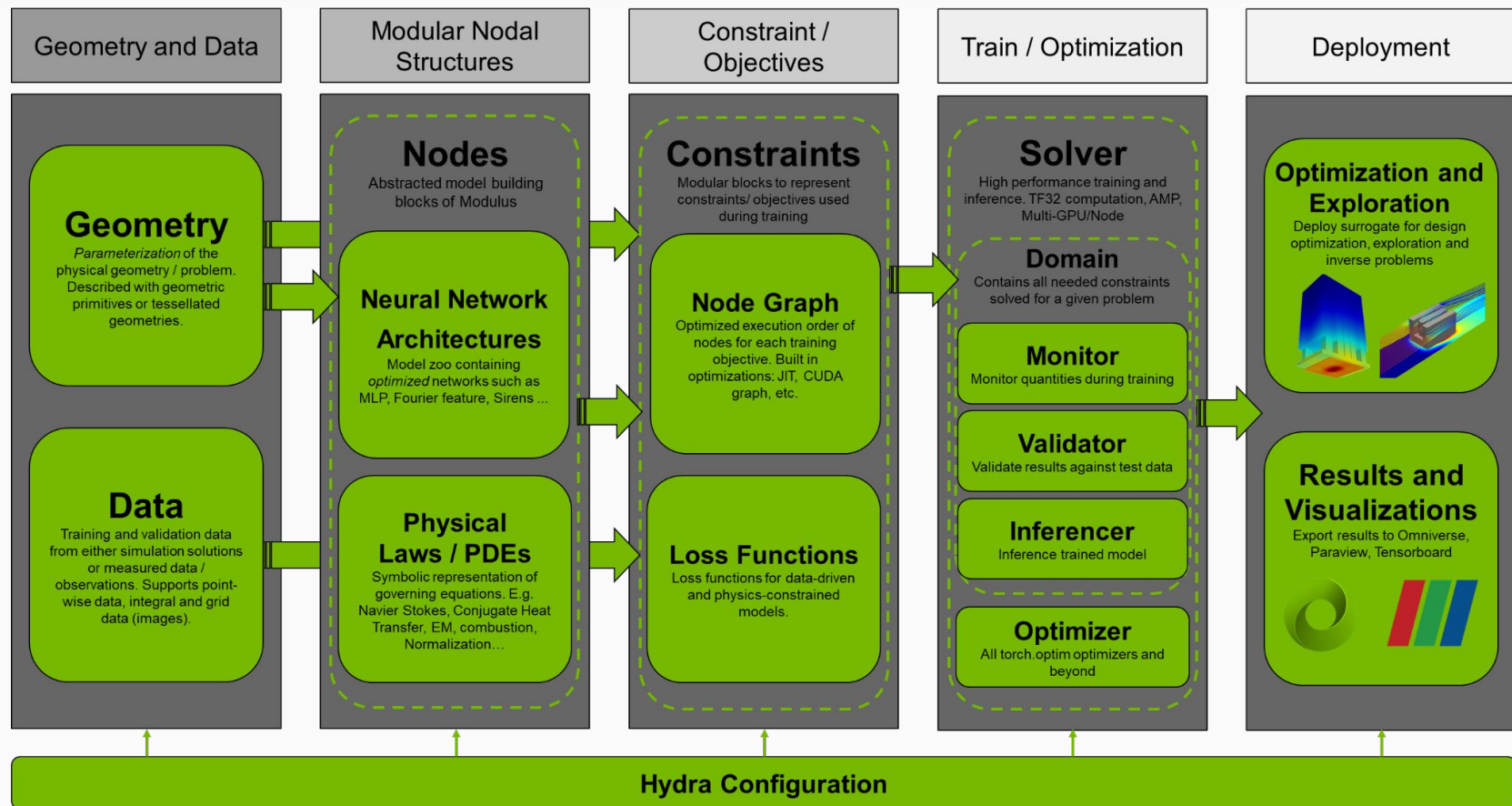
Collection of training/inference
examples for Phys-ML applications

NVIDIA Modulus Sym

- Framework para modelos sustitutos basados en IA mediante expresiones físicas simbólicas
- Abstracción de alto nivel para entrenar modelos informados por física (via API)
- Se pueden combinar numerosos “nodos” o “modelos” para aprender diferentes ecuaciones
- Admite paradigmas de paralelización sin dificultades



NVIDIA Modulus Sym

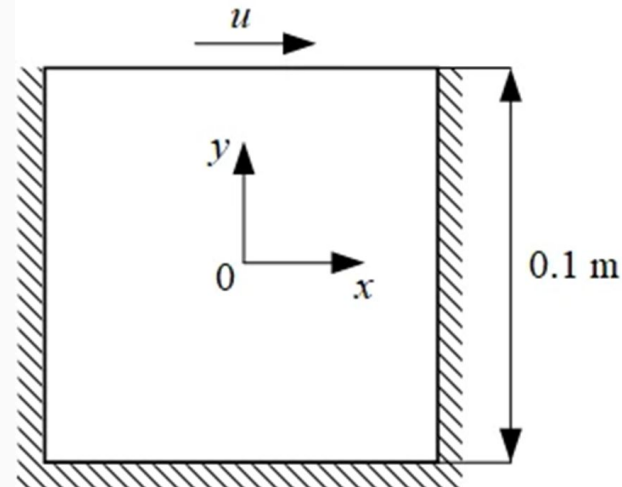


NVIDIA Modulus Sym - Ejemplo: lid driven cavity

Creación del nodo PDE:

```
from modulus.sym.eq.pdes.navier_stokes import NavierStokes

# make list of nodes to unroll graph on
ns = NavierStokes(nu=0.01, rho=1.0, dim=2, time=False)
```



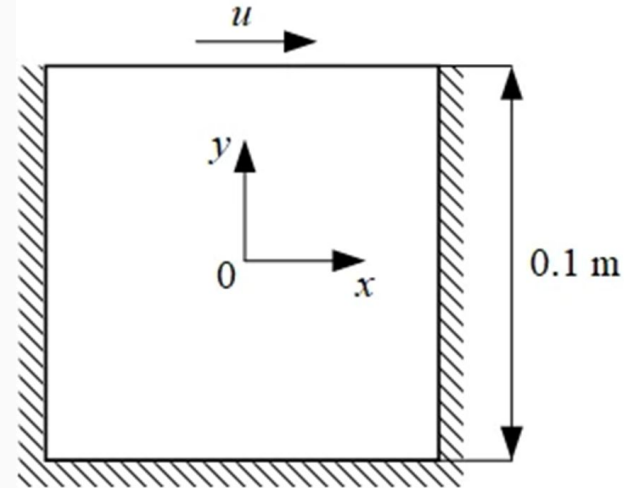
$$\frac{\partial u}{\partial x} + \frac{\partial v}{\partial y} = 0$$
$$u \frac{\partial u}{\partial x} + v \frac{\partial u}{\partial y} = -\frac{\partial p}{\partial x} + \nu \left(\frac{\partial^2 u}{\partial x^2} + \frac{\partial^2 u}{\partial y^2} \right)$$
$$u \frac{\partial v}{\partial x} + v \frac{\partial v}{\partial y} = -\frac{\partial p}{\partial y} + \nu \left(\frac{\partial^2 v}{\partial x^2} + \frac{\partial^2 v}{\partial y^2} \right)$$

NVIDIA Modulus Sym - Ejemplo: lid driven cavity

Creación del nodo Red Neuronal:

```
from modulus.sym.hydra import instantiate_arch
from modulus.sym.key import Key

flow_net = instantiate_arch(
    input_keys=[Key("x"), Key("y")],
    output_keys=[Key("u"), Key("v"), Key("p")],
    cfg=cfg.arch.fully_connected,
)
nodes = ns.make_nodes() + [flow_net.make_node(name="flow_network")]
```



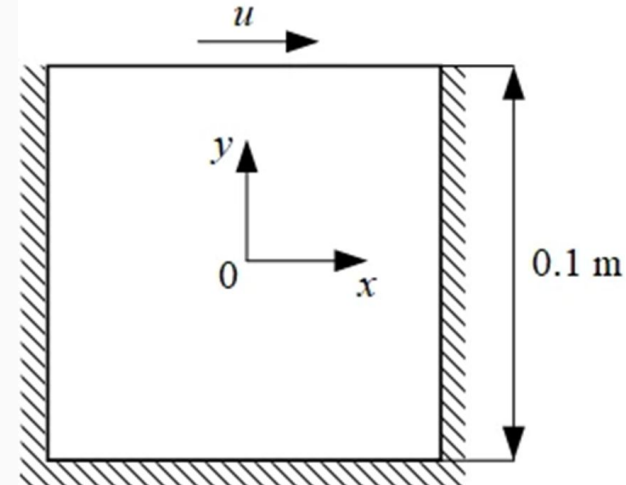
$$\frac{\partial u}{\partial x} + \frac{\partial v}{\partial y} = 0$$
$$u \frac{\partial u}{\partial x} + v \frac{\partial u}{\partial y} = -\frac{\partial p}{\partial x} + \nu \left(\frac{\partial^2 u}{\partial x^2} + \frac{\partial^2 u}{\partial y^2} \right)$$
$$u \frac{\partial v}{\partial x} + v \frac{\partial v}{\partial y} = -\frac{\partial p}{\partial y} + \nu \left(\frac{\partial^2 v}{\partial x^2} + \frac{\partial^2 v}{\partial y^2} \right)$$

NVIDIA Modulus Sym - Ejemplo: lid driven cavity

Creación del nodo Geometría:

```
from sympy import Symbol, Eq, Abs
from modulus.sym.geometry.primitives_2d import Rectangle

# make geometry
height = 0.1
width = 0.1
x, y = Symbol("x"), Symbol("y")
rec = Rectangle((-width / 2, -height / 2), (width / 2, height / 2))
```



$$\frac{\partial u}{\partial x} + \frac{\partial v}{\partial y} = 0$$
$$u \frac{\partial u}{\partial x} + v \frac{\partial u}{\partial y} = -\frac{\partial p}{\partial x} + \nu \left(\frac{\partial^2 u}{\partial x^2} + \frac{\partial^2 u}{\partial y^2} \right)$$
$$u \frac{\partial v}{\partial x} + v \frac{\partial v}{\partial y} = -\frac{\partial p}{\partial y} + \nu \left(\frac{\partial^2 v}{\partial x^2} + \frac{\partial^2 v}{\partial y^2} \right)$$

NVIDIA Modulus Sym - Ejemplo: lid driven cavity

Definición de la instancia Domain y restricciones a la PDE:

```
from modulus.sym.domain import Domain

# make ldc domain
ldc_domain = Domain()
```

El objeto Domain contiene toda la información de restricciones y referidas a los conjuntos de validación, inferencia, etc

```
from modulus.sym.domain.constraint import PointwiseInteriorConstraint

# interior
interior = PointwiseInteriorConstraint(
    nodes=nodes,
    geometry=rec,
    outvar={"continuity": 0, "momentum_x": 0, "momentum_y": 0},
    batch_size=cfg.batch_size.Interior,
    lambda_weighting={
        "continuity": Symbol("sdf"),
        "momentum_x": Symbol("sdf"),
        "momentum_y": Symbol("sdf"),
    },
)
ldc_domain.add_constraint(interior, "interior")
```

Cada ecuación en cada punto se encuentra pesada por su distancia a los bordes mediante la “signed distance field” (SDF) de la geometría

$$L_{continuity} = \frac{V}{N} \sum_{i=0}^N (0 - continuity(x_i, y_i))^2$$

$$L_{momentum_x} = \frac{V}{N} \sum_{i=0}^N (0 - momentum_x(x_i, y_i))^2$$

$$L_{momentum_y} = \frac{V}{N} \sum_{i=1}^n (0 - momentum_y(x_i, y_i))^2$$

NVIDIA Modulus Sym - Ejemplo: lid driven cavity

Definición de condiciones de borde:

```
from modulus.sym.domain.constraint import PointwiseBoundaryConstraint

# top wall
top_wall = PointwiseBoundaryConstraint(
    nodes=nodes,
    geometry=rec,
    outvar={"u": 1.0, "v": 0},
    batch_size=cfg.batch_size.TopWall,
    lambda_weighting={"u": 1.0 - 20 * Abs(x), "v": 1.0},
    criteria=Eq(y, height / 2),
)
ldc_domain.add_constraint(top_wall, "top_wall")

# no slip
no_slip = PointwiseBoundaryConstraint(
    nodes=nodes,
    geometry=rec,
    outvar={"u": 0, "v": 0},
    batch_size=cfg.batch_size.NoSlip,
    criteria=y < height / 2,
)
ldc_domain.add_constraint(no_slip, "no_slip")
```

NVIDIA Modulus Sym - Ejemplo: lid driven cavity

Definición de conjunto de validación:

```
from modulus.sym.hydra import to_absolute_path
from modulus.sym.domain.validator import PointwiseValidator
from modulus.sym.utils.io import (
    csv_to_dict,
    ValidatorPlotter,
    InferencerPlotter,
)

# add validator
mapping = {"Points:0": "x", "Points:1": "y", "U:0": "u", "U:1": "v", "p": "p"}
openfoam_var = csv_to_dict(
    to_absolute_path("openfoam/cavity_uniformVel0.csv"), mapping
)
openfoam_var["x"] += -width / 2 # center OpenFoam data
openfoam_var["y"] += -height / 2 # center OpenFoam data
openfoam_invar_numpy = {
    key: value for key, value in openfoam_var.items() if key in ["x", "y"]
}
openfoam_outvar_numpy = {
    key: value for key, value in openfoam_var.items() if key in ["u", "v"]
}
openfoam_validator = PointwiseValidator(
    nodes=nodes,
    invar=openfoam_invar_numpy,
    true_outvar=openfoam_outvar_numpy,
    batch_size=1024,
    plotter=ValidatorPlotter(),
)
ldc_domain.add_validator(openfoam_validator)
```

El conjunto de validación se debe suministrar a partir de datos ground-truth de simulaciones complementarias

NVIDIA Modulus Sym - Ejemplo: lid driven cavity

Entrenamiento y postproceso:

```
import os
from modulus.solver import Solver

# make solver
slv = Solver(cfg, ldc_domain)

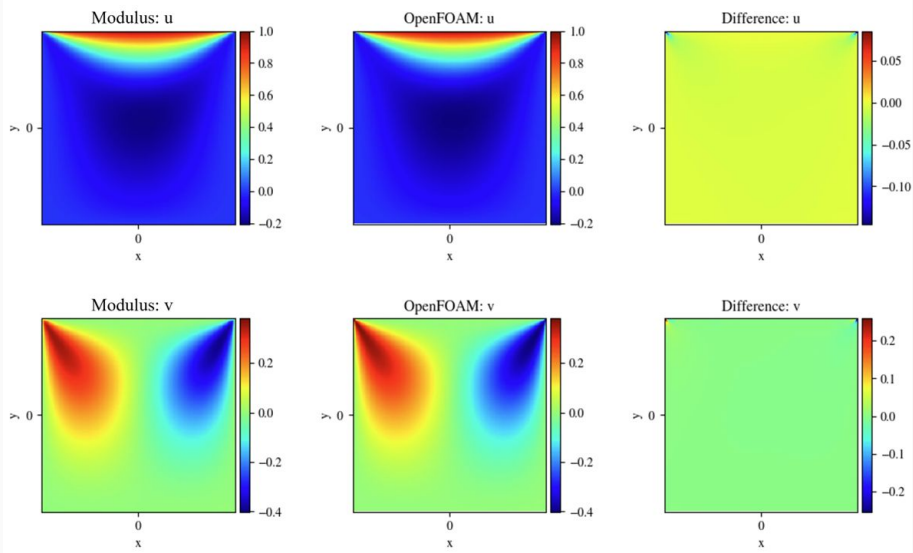
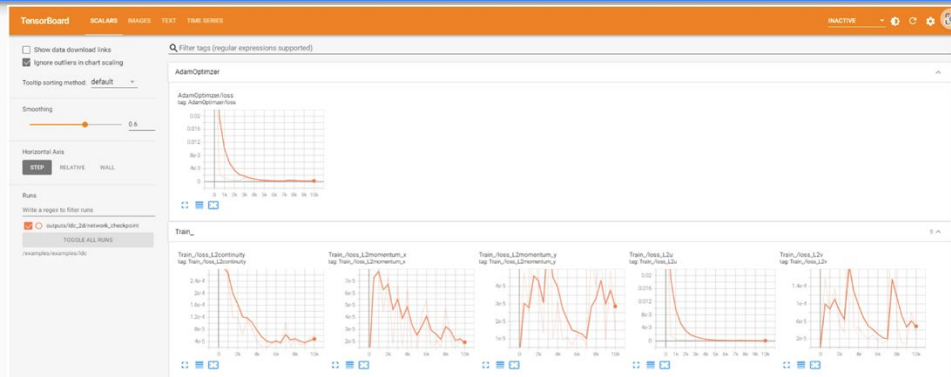
# start solver
slv.solve()
```

Ejecución del script de python:

```
python ldc_2d.py
```

Tensorboard para visualizar resultados

```
tensorboard --logdir=./ --port=7007
```



Temas que vienen en el futuro

- PINN + GANs (Yang et al., 2020)
- Kolmogorov-Arnold networks (KANs) (Liu et al., 2024)
- PhyGNNs (Jiang et al., 2023)
- DD-FEM models and Foundational models for Science and Engineering (Choi et al., 2025)
- Y muchas otras cosas más por descubrir....



¿Dudas o
preguntas?